

Université Mohammed V de Rabat
École Nationale Supérieure d'Arts et Métiers

Filière INDIA – 2ème année – S4

Année Universitaire : 2025/2026

**Gestion des Lits et des
Flux Hospitaliers**

Architecture Microservices avec Spring Boot & Docker

Mini Projet – Architecture Distribuée

Réalisé par :

Aymane ISSAMI
Houssam KICHCHOU

Encadré par :

Pr. A. EL QADI

Avril 2026

Contents

1	Introduction	2
1.1	Contexte du projet	2
1.2	Objectifs	2
1.3	Acteurs du système	3
2	Architecture du Système	4
2.1	Vue d'ensemble	4
2.2	Diagramme d'architecture	4
2.3	Description des microservices	5
2.4	Technologies utilisées	5
3	Modèle de Données	6
3.1	Entité Bed (Lit)	6
3.1.1	Cycle de vie d'un lit	6
3.2	Entité Patient	7
3.2.1	Trajectoire du patient	8
3.3	Entité PatientTransfer	8
3.4	Entité OccupancyRecord	9
3.5	Entité Notification	9
4	Flux de Données et Communication Inter-services	10
4.1	Flux principaux	10
4.2	Scénario : Admission complète d'un patient	10
4.3	Communication inter-services	11
5	API REST	12
5.1	Bed Management Service – /api/beds	12
5.1.1	Exemple de requête – Création d'un lit	12
5.2	Patient Flow Service – /api/patients	12
5.3	Capacity Prediction Service – /api/capacity	13
5.3.1	Algorithme de prédiction	13
5.4	Notification Ops Service – /api/notifications	14
5.5	Dashboard Service – /api/dashboard	14
6	Déploiement avec Docker	15
6.1	Stratégie de conteneurisation	15
6.2	Dockerfile type	15
6.3	Ordre de démarrage	15
6.4	Commandes de déploiement	16

6.5	Ports exposés	16
7	Gestion des Conflits d'Affectation	17
7.1	Problématique	17
7.2	Stratégies implémentées	17
7.2.1	Machine à états stricte	17
7.2.2	Vérification atomique	17
7.2.3	Notifications proactives	17
8	Indicateurs de Performance (KPIs)	18
9	Tests et Validation	19
9.1	Collection Postman	19
9.2	Documentation Swagger	19
9.3	Scénario de test intégré	19
10	Conclusion	21

Chapter 1

Introduction

1.1 Contexte du projet

Dans le cadre du module d'Architecture Distribuée, ce mini projet vise à concevoir et développer un système de gestion des lits et des flux hospitaliers en utilisant une architecture microservices. Le système permet d'optimiser l'affectation des lits, le transfert entre services et la planification des sorties pour réduire les temps d'attente, notamment aux urgences.

La gestion efficace des lits hospitaliers représente un enjeu majeur pour les établissements de santé. Une mauvaise gestion peut entraîner :

- Des temps d'attente excessifs aux urgences
- Une sous-utilisation ou surcharge de certains services
- Des conflits d'affectation entre services
- Un manque de visibilité sur la capacité disponible

1.2 Objectifs

1. **Gestion des lits** : Suivre l'état de chaque lit (libre, occupé, réservé, en nettoyage) avec un cycle de vie complet
2. **Flux patients** : Gérer la trajectoire des patients de l'admission à la sortie, incluant les transferts inter-services
3. **Prédiction de capacité** : Anticiper les besoins en lits via des règles de prédiction
4. **Notifications opérationnelles** : Alerter les équipes (brancardage, nettoyage, transferts)
5. **Tableau de bord** : Fournir une vue agrégée en temps réel avec des indicateurs clés

1.3 Acteurs du système

Acteur	Rôle
Urgentistes / Médecins régulateurs	Gestion des admissions d'urgence, décisions de transfert
Cadre de santé / Gestionnaire de lits	Affectation des lits, suivi de la disponibilité
Brancardiers / Logistique interne	Transport des patients, nettoyage des lits
Direction	Consultation du tableau de bord opérationnel

Table 1.1: Acteurs du système

Chapter 2

Architecture du Système

2.1 Vue d'ensemble

Le système est construit selon une architecture microservices, où chaque service est indépendant, déployable séparément et communique via des APIs REST. Un service de découverte (Eureka) assure le registre des services et un API Gateway fournit un point d'entrée unique.

2.2 Diagramme d'architecture

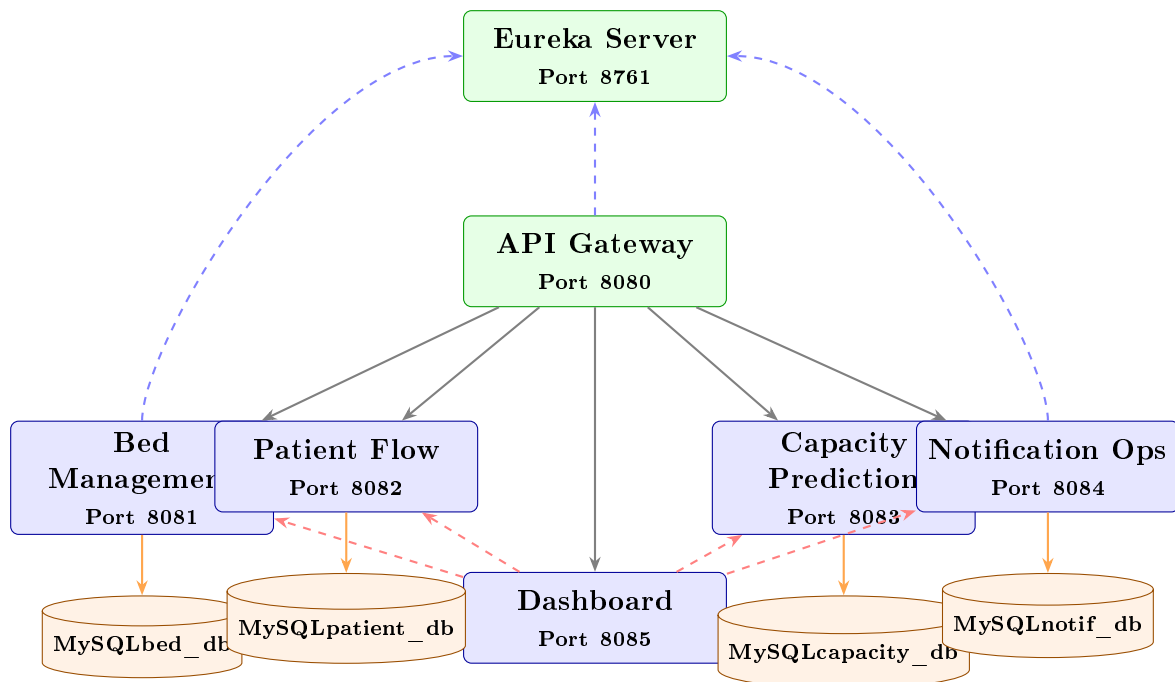


Figure 2.1: Diagramme d'architecture microservices

2.3 Description des microservices

Service	Port	Rôle
discovery-service	8761	Registre Eureka – découverte automatique des services
gateway-service	8080	Point d'entrée unique, routage, CORS
bed-management-service	8081	CRUD lits, cycle de vie des statuts, affectation
patient-flow-service	8082	Admission, transferts, sorties, trajectoire patient
capacity-prediction-service	8083	Prédiction d'occupation, tendances, niveaux de risque
notification-ops-service	8084	Alertes nettoyage, brancardage, transferts, capacité
dashboard-service	8085	Agrégation temps réel, KPIs, tableau de bord

Table 2.1: Liste des microservices

2.4 Technologies utilisées

Technologie	Usage
Java 21	Langage de programmation
Spring Boot 3.2.4	Framework applicatif
Spring Cloud 2023.0.0	Eureka Discovery, Gateway
Spring Data JPA	Couche de persistance (Hibernate)
MySQL 8.0	Base de données relationnelle (production)
H2 Database	Base de données embarquée (développement)
Docker & Docker Compose	Conteneurisation et orchestration
OpenAPI 3 / Swagger	Documentation interactive des APIs
Maven	Gestion des dépendances et build
WebClient (WebFlux)	Communication inter-services

Table 2.2: Stack technologique

Chapter 3

Modèle de Données

3.1 Entité Bed (Lit)

L'entité **Bed** représente un lit hospitalier avec son état actuel et ses métadonnées.

Champ	Type	Description
id	Long	Identifiant unique (auto-généré)
bedNumber	String	Numéro du lit (unique)
ward	String	Service hospitalier (Urgences, Cardiologie...)
roomNumber	String	Numéro de la chambre
floor	Integer	Étage
bedType	BedType	STANDARD, ICU, PEDIATRIC, MATERNITY
status	BedStatus	FREE, OCCUPIED, RESERVED, CLEANING
currentPatientId	Long	Référence au patient actuel (nullable)
lastStatusChange	LocalDateTime	Horodatage du dernier changement
notes	String	Notes libres

Table 3.1: Modèle de données – Bed

3.1.1 Cycle de vie d'un lit

Le statut d'un lit suit une machine à états précise :

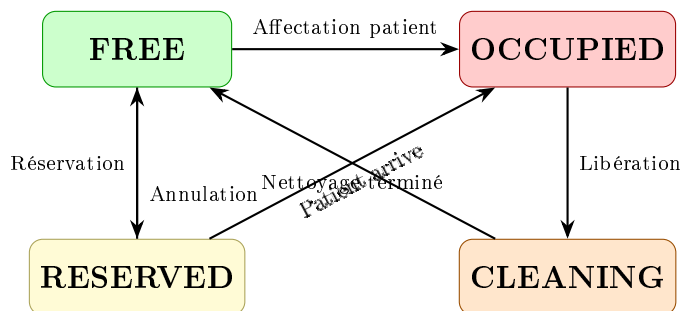


Figure 3.1: Machine à états du statut d'un lit

Transitions valides :

- FREE → OCCUPIED : Affectation directe d'un patient
- FREE → RESERVED : Réservation pour une admission programmée
- RESERVED → OCCUPIED : Le patient arrive et occupe le lit réservé
- RESERVED → FREE : Annulation de la réservation
- OCCUPIED → CLEANING : Le patient quitte le lit (sortie ou transfert)
- CLEANING → FREE : Le nettoyage est terminé, le lit est disponible

Toute autre transition est rejetée par le service avec un message d'erreur explicite.

3.2 Entité Patient

Champ	Type	Description
id	Long	Identifiant unique
firstName / lastName	String	Nom et prénom
dateOfBirth	LocalDate	Date de naissance
gender	String	Genre (M/F)
nationalId	String	CIN / identifiant national (unique)
contactPhone	String	Téléphone
emergencyContact	String	Contact d'urgence
currentStatus	PatientStatus	REGISTERED, ADMITTED, IN_TRANSFER, DISCHARGED, EMERGENCY
currentWard	String	Service actuel
admissionDate	LocalDateTime	Date d'admission
dischargeDate	LocalDateTime	Date de sortie
medicalNotes	String	Notes médicales

Table 3.2: Modèle de données – Patient

3.2.1 Trajectoire du patient

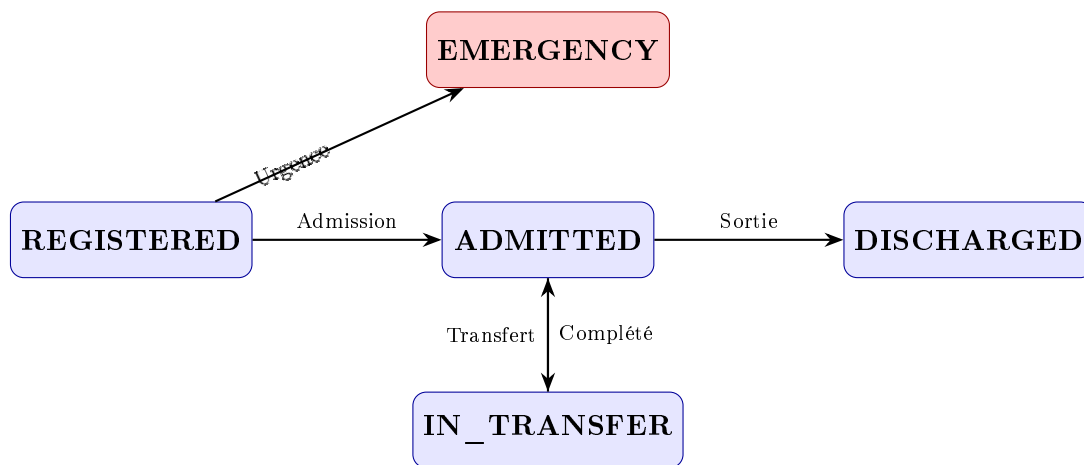


Figure 3.2: Diagramme d'états du patient

3.3 Entité PatientTransfer

Champ	Type	Description
id	Long	Identifiant unique
patientId	Long	Référence au patient
fromWard	String	Service d'origine
toWard	String	Service de destination
transferDate	LocalDateTime	Date du transfert
reason	String	Motif du transfert
authorizedBy	String	Médecin autorisant
status	TransferStatus	PENDING, IN_PROGRESS, COMPLETED, CANCELLED

Table 3.3: Modèle de données – PatientTransfer

3.4 Entité OccupancyRecord

Champ	Type	Description
id	Long	Identifiant unique
ward	String	Service hospitalier
date	LocalDate	Date de l'enregistrement
hour	Integer	Heure (0–23)
totalBeds	int	Nombre total de lits
occupiedBeds	int	Nombre de lits occupés
occupancyRate	double	Taux d'occupation
recordedAt	LocalDateTime	Horodatage

Table 3.4: Modèle de données – OccupancyRecord

3.5 Entité Notification

Champ	Type	Description
id	Long	Identifiant unique
type	NotificationType	BED_RELEASE, CLEANING_REQUIRED, TRANSFER_ALERT, etc.
title	String	Titre de la notification
message	String	Corps du message
targetWard	String	Service concerné
priority	Priority	LOW, MEDIUM, HIGH, URGENT
status	NotificationStatus	PENDING, SENT, READ, ARCHIVED
createdAt	LocalDateTime	Date de création

Table 3.5: Modèle de données – Notification

Chapter 4

Flux de Données et Communication Inter-services

4.1 Flux principaux

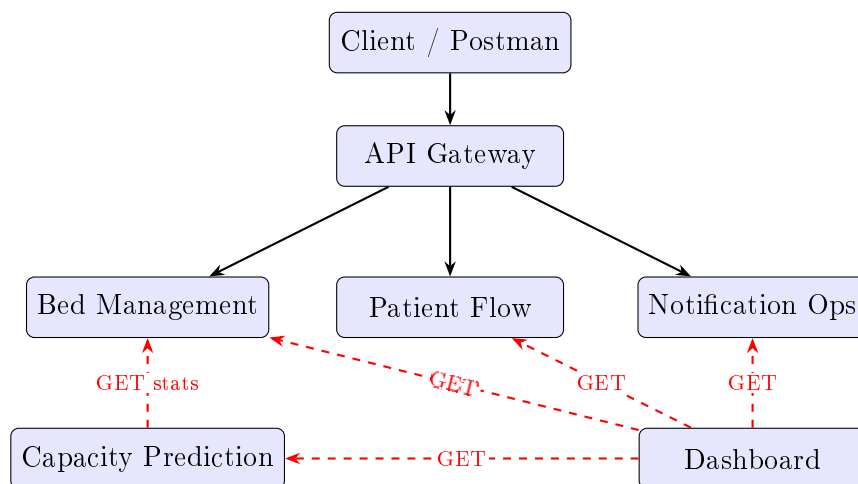


Figure 4.1: Flux de communication entre microservices

4.2 Scénario : Admission complète d'un patient

Voici le flux complet d'un scénario typique :

1. **Enregistrement** : POST /api/patients – Création du dossier patient (statut: REGISTERED)
2. **Vérification des lits** : GET /api/beds?ward=Urgences&status=FREE – Recherche d'un lit disponible
3. **Admission** : POST /api/patients/{id}/admit – Le patient est admis (statut: ADMITTED)
4. **Affectation du lit** : POST /api/beds/assign – Le lit passe en OCCUPIED
5. **Notification** : POST /api/notifications – Alerte au service concerné

6. **Transfert** (si nécessaire) : `POST /api/patients/{id}/transfer` – Demande de transfert
7. **Libération du lit** : `POST /api/beds/{id}/release` – Le lit passe en CLEANING
8. **Nettoyage** : `PUT /api/beds/{id}/status` – Le lit repasse en FREE
9. **Sortie** : `POST /api/patients/{id}/discharge` – Le patient sort (statut: DISCHARGED)

4.3 Communication inter-services

La communication entre microservices se fait via **WebClient** (Spring WebFlux), un client HTTP réactif non-bloquant. Les services concernés sont :

- **capacity-prediction-service** : Appelle `bed-management-service` pour obtenir les statistiques d'occupation actuelles
- **dashboard-service** : Appelle les 4 autres services pour agréger les données en temps réel, avec des fallbacks en cas d'indisponibilité

Chapter 5

API REST

Chaque microservice expose une API REST documentée via OpenAPI 3 (Swagger). Les APIs sont accessibles directement ou via le Gateway (port 8080).

5.1 Bed Management Service – /api/beds

Méthode	Endpoint	Description
GET	/api/beds	Lister tous les lits (filtres optionnels : ward, status)
GET	/api/beds/{id}	Obtenir un lit par ID
POST	/api/beds	Créer un nouveau lit
PUT	/api/beds/{id}/status	Modifier le statut d'un lit
DELETE	/api/beds/{id}	Supprimer un lit
POST	/api/beds/assign	Affecter un lit à un patient
POST	/api/beds/{id}/release	Libérer un lit (passe en CLEANING)
GET	/api/beds/statistics	Statistiques globales et par service
GET	/api/beds/assignments/{bedId}	Historique des affectations

5.1.1 Exemple de requête – Création d'un lit

```
1 {  
2   "bedNumber": "URG-101",  
3   "ward": "Urgences",  
4   "roomNumber": "101",  
5   "floor": 1,  
6   "bedType": "STANDARD",  
7   "notes": "Lit standard aux urgences"  
8 }
```

Listing 5.1: POST /api/beds

5.2 Patient Flow Service – /api/patients

Méthode	Endpoint	Description
GET	/api/patients	Lister les patients
GET	/api/patients/{id}	Obtenir un patient
POST	/api/patients	Enregistrer un patient
POST	/api/patients/{id}/admit	Admettre un patient
POST	/api/patients/{id}/transfer	Initier un transfert
POST	/api/patients/transfers/{id}/complete	Compléter un transfert
POST	/api/patients/{id}/discharge	Sortie du patient
GET	/api/patients/{id}/transfers	Historique des transferts
GET	/api/patients/statistics	Statistiques des flux

5.3 Capacity Prediction Service – /api/capacity

Méthode	Endpoint	Description
GET	/api/capacity/current	Capacité actuelle globale
GET	/api/capacity/prediction/{ward}	Prédiction pour un service
GET	/api/capacity/predictions	Toutes les prédictions
POST	/api/capacity/record	Enregistrer un snapshot d'occupation

5.3.1 Algorithme de prédiction

Le service utilise un algorithme basé sur des règles simples :

- Récupération des enregistrements d'occupation des dernières 48h
- Calcul de la tendance : moyenne des 24 dernières heures vs 24 heures précédentes
- Classification de la tendance :
 - Augmentation > 5% → **INCREASING**
 - Diminution > 5% → **DECREASING**
 - Sinon → **STABLE**
- Évaluation du niveau de risque :
 - Taux > 90% → **CRITICAL**
 - Taux > 75% → **HIGH**
 - Taux > 50% → **MEDIUM**
 - Sinon → **LOW**
- Génération de recommandations adaptées au niveau de risque

5.4 Notification Ops Service – /api/notifications

Méthode	Endpoint	Description
GET	/api/notifications	Lister les notifications
POST	/api/notifications	Créer une notification
PUT	/api/notifications/{id}/read	Marquer comme lue
GET	/api/notifications/ward/{ward}/unread	Non lues par service
GET	/api/notifications/statistics	Statistiques
POST	/api/notifications/bed-release	Alerte libération de lit
POST	/api/notifications/cleaning	Alerte nettoyage requis
POST	/api/notifications/transfer-alert	Alerte transfert

5.5 Dashboard Service – /api/dashboard

Méthode	Endpoint	Description
GET	/api/dashboard	Tableau de bord complet
GET	/api/dashboard/kpis	Indicateurs clés uniquement

Les KPIs calculés incluent :

- Taux d'occupation global
- Nombre de lits disponibles
- Taux de transfert
- Temps moyen de rotation des lits
- Nombre de notifications non lues

Chapter 6

Déploiement avec Docker

6.1 Stratégie de conteneurisation

Chaque microservice est packagé dans un conteneur Docker indépendant. Le fichier `docker-compose.yml` orchestre l'ensemble avec :

- **4 bases MySQL** : Une par service nécessitant de la persistance (bed, patient, capacity, notification)
- **7 services applicatifs** : Discovery, Gateway + 5 services métier
- **Health checks** : Vérification de l'état des services avant démarrage des dépendants
- **Réseau dédié** : `hospital-network` en mode bridge

6.2 Dockerfile type

```
1 FROM eclipse-temurin:17-jdk-alpine
2 VOLUME /tmp
3 COPY target/*.jar app.jar
4 EXPOSE 8081
5 ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Listing 6.1: Dockerfile d'un microservice

6.3 Ordre de démarrage

1. **Bases de données MySQL** (4 instances en parallèle)
2. **discovery-service** (Eureka) – attend le health check
3. **Services métier** (bed, patient, capacity, notification) – attendent Eureka + leur MySQL
4. **dashboard-service** – attend tous les services métier
5. **gateway-service** – attend Eureka

6.4 Commandes de déploiement

```

1 # Compiler tous les services
2 mvn clean package -DskipTests
3
4 # Lancer tous les conteneurs
5 docker compose up --build
6
7 # Lancer en arriere-plan
8 docker compose up --build -d
9
10 # Voir les logs d'un service
11 docker compose logs -f bed-management-service
12
13 # Arrêter tous les conteneurs
14 docker compose down
15
16 # Arrêter et supprimer les volumes
17 docker compose down -v

```

Listing 6.2: Commandes pour déployer le système

6.5 Ports exposés

Service	Port interne	Port exposé
MySQL (bed)	3306	3307
MySQL (patient)	3306	3308
MySQL (capacity)	3306	3309
MySQL (notification)	3306	3310
Eureka	8761	8761
Gateway	8080	8080
Bed Management	8081	8081
Patient Flow	8082	8082
Capacity Prediction	8083	8083
Notification Ops	8084	8084
Dashboard	8085	8085

Table 6.1: Mapping des ports Docker

Chapter 7

Gestion des Conflits d'Affectation

7.1 Problématique

Dans un hôpital, plusieurs services peuvent demander le même lit simultanément. Les conflits typiques sont :

- Deux médecins tentent d'affecter le même lit à des patients différents
- Un lit réservé est demandé en urgence par un autre service
- Un patient est transféré mais le lit de destination n'est pas encore libéré

7.2 Stratégies implémentées

7.2.1 Machine à états stricte

Le service `bed-management-service` implémente une machine à états qui rejette toute transition invalide. Par exemple :

- Un lit `OCCUPIED` ne peut pas être affecté à un autre patient sans libération préalable
- Un lit `CLEANING` ne peut pas être affecté directement

7.2.2 Vérification atomique

L'affectation d'un lit (`POST /api/beds/assign`) vérifie que le lit est `FREE` avant de le passer en `OCCUPIED`. Grâce à la transaction JPA, cette opération est atomique au niveau de la base de données.

7.2.3 Notifications proactives

Le service de notifications alerte automatiquement les équipes concernées lors de :

- Libération d'un lit (alerte nettoyage)
- Fin de nettoyage (lit disponible)
- Transfert en cours (préparation du service de destination)
- Capacité critique (alerte direction)

Chapter 8

Indicateurs de Performance (KPIs)

Le `dashboard-service` calcule et expose les indicateurs suivants :

KPI	Unité	Seuils	Description
Taux d'occupation	%	< 75% OK, > 90% critique	Ratio lits occupés / total
Lits disponibles	nombre	–	Nombre de lits au statut FREE
Taux de transfert	%	–	Ratio patients transférés / admis
Rotation des lits	heures	< 4h bon	Temps moyen entre libération et réoccupation
Notifications non lues	nombre	> 20 alerte	Notifications en attente de lecture

Table 8.1: Indicateurs clés de performance

Chaque KPI est retourné avec :

- **name** : Nom de l'indicateur
- **value** : Valeur actuelle
- **unit** : Unité de mesure
- **trend** : Tendence (UP, DOWN, STABLE)
- **status** : État (GOOD, WARNING, CRITICAL)

Chapter 9

Tests et Validation

9.1 Collection Postman

Une collection Postman complète est fournie dans le dossier `postman/` avec plus de 40 requêtes organisées par service :

1. **Bed Management** (14 requêtes) : CRUD, affectation, libération, statistiques
2. **Patient Flow** (12 requêtes) : Enregistrement, admission, transfert, sortie
3. **Capacity Prediction** (4 requêtes) : Capacité, prédictions, enregistrement
4. **Notification Ops** (8 requêtes) : CRUD, alertes rapides, statistiques
5. **Dashboard** (2 requêtes) : Tableau de bord, KPIs
6. **Via Gateway** (5 requêtes) : Vérification du routage

9.2 Documentation Swagger

Chaque service dispose d'une interface Swagger accessible à :

- `http://localhost:8081/swagger-ui.html` – Bed Management
- `http://localhost:8082/swagger-ui.html` – Patient Flow
- `http://localhost:8083/swagger-ui.html` – Capacity Prediction
- `http://localhost:8084/swagger-ui.html` – Notification Ops
- `http://localhost:8085/swagger-ui.html` – Dashboard

9.3 Scénario de test intégré

Pour valider le système de bout en bout :

1. Créer 4 lits dans différents services (Urgences, Cardiologie, Pédiatrie, Maternité)
2. Créer 2 patients

3. Admettre le premier patient aux Urgences
4. Affecter le lit URG-101 au patient
5. Vérifier que le lit est en statut OCCUPIED
6. Initier un transfert du patient vers la Cardiologie
7. Compléter le transfert
8. Libérer le lit aux Urgences (passe en CLEANING)
9. Vérifier les notifications générées
10. Consulter le tableau de bord pour les statistiques globales
11. Sortir le patient (discharge)
12. Vérifier les KPIs finaux

Chapter 10

Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux de l'architecture distribuée à travers un cas d'usage concret : la gestion des lits hospitaliers. Les principaux acquis sont :

- **Architecture microservices** : Décomposition fonctionnelle en 7 services indépendants, chacun avec sa propre base de données et son cycle de vie
- **Service Discovery** : Utilisation d'Eureka pour l'enregistrement et la découverte dynamique des services
- **API Gateway** : Point d'entrée unique avec routage et CORS
- **Communication inter-services** : WebClient réactif pour les appels HTTP entre services
- **Conteneurisation** : Docker et Docker Compose pour le déploiement reproductible
- **Documentation** : APIs documentées avec OpenAPI/Swagger

Améliorations possibles

- Ajout d'un système d'authentification (JWT / OAuth2)
- Implémentation d'un circuit breaker (Resilience4j)
- Utilisation de Kafka pour la communication événementielle entre services
- Intégration de modèles ML pour la prédiction de capacité
- Ajout d'une interface web (Angular / React)
- Monitoring avec Prometheus et Grafana